Complexidade Média

Busca em Lista Ordenada

Busca em Lista Ordenada

```
Busca_lista_ord ( S[1,..,n], x )
{
  i = 1;
  enquanto (i ≤ n) e (x < s_i ) faça i = i+1;
  se x=s_i então retorne (i);
  senão retorne (-1);
}
```



S={s_1, s_2,...,s_n} uma seqüência. Um valor x

Busca em Lista Ordenada

O (n) ??

A questão que se coloca é qual seria a complexidade média para uma operação real em que esse algoritmo fosse aplicado a diversas instâncias do problema?

Na média, o que aconteceria?

Complexidade de Caso Médio

$$CM = \sum_{E_i \in S} p(E_i)\, t(E_i)$$

S é o conjunto de todas as entradas possíveis

E_i é um elemento de S

$p(E_i)$ é a probabilidade de ocorrência da entrada E_i

$t(E_i)$ é o tempo requerido pela entrada E_i para ser processada

Busca em Lista Ordenada

E_i representam os casos de sucesso da busca, isto é, os casos em que x é igual a alguma chave s_i

As entradas R_i, $i = 0, 1, .., n$, representam o fracasso da busca, ou seja, as entradas:

$$R_0 = \{x \mid x < s_1\};$$

$$R_1 = \{x \mid s_1 < x < s_2\};$$

$$R_2 = \{x \mid s_2 < x < s_3\};$$

$$R_n = \{x \mid x > s_n\};$$

$$CM = \sum_{i=0}^{n} p(R_i)\, t(R_i) + \sum_{i=1}^{n} p(E_i)\, t(E_i)$$

Busca em Lista Ordenada

Considerando que a probabilidade de $x \in S$ é modelada através de uma distribuição uniforme, e que seu valor será igual a q, então:

Probabilidade de sucesso = q

Probabilidade de fracasso = $1 - q$

O número de passos que o algoritmo desenvolve em cada uma das duas hipóteses anteriores é, respectivamente:

$$t(E_i) = i, \quad 1 \leq i \leq n;$$

$$t(R_i) = i + 1, \quad 0 \leq i \leq n;$$

Busca em Lista Ordenada

$$p(E_i) = \frac{q}{n} \qquad t(E_i) = i$$

$$p(R_i) = \frac{1-q}{n+1} \qquad t(R_i) = i+1$$

Busca em Lista Ordenada

$$CM = \sum_{i=0}^{n} p(R_i)\, t(R_i) + \sum_{i=1}^{n} p(E_i)\, t(E_i)$$

$$= \sum_{i=0}^{n} \frac{(1-q)}{n+1}(i+1) + \sum_{i=1}^{n} \frac{q}{n} i$$

$$= \frac{(1-q)}{n+1} \sum_{i=0}^{n} (i+1) + \frac{q}{n} \sum_{i=1}^{n} i$$

Busca em Lista Ordenada

$$CM = \frac{(1-q)}{n+1}\sum_{i=0}^{n}(i+1) + \frac{q}{n}\sum_{i=1}^{n} i$$

$$CM = \frac{n-q+2}{2}$$

Busca em Lista Ordenada

$$q = 0$$

$$CM = \frac{n+2}{2} = \frac{n}{2} + 1$$

$$q = 1$$

$$CM = \frac{n+1}{2}$$

Quicksort

```
Quicksort(L,1,n) //lista L com tamanho p de inteiros
   k ← pivô(L)     // inicializa k com o índice do pivô
   se k ≠ 0 então
     Troca(L[1],L[k])
     j ← Particiona(L,1,n)
     L1 ← Quicksort(L,1,j-1)
     L2 ← Quicksort(L,j,n)
     L  ← Concatena(L1,L2)
  retorna(L)
```

Quicksort

pivô(*L*) – retorna o índice do maior número entre os dois primeiros diferentes em *L*

Se todos os elementos de *L* forem iguais, retorna 0 indicando que a lista já está ordenada.

Quicksort

Particiona(*L*,1,*n*)

$v \leftarrow L[1]$; $i \leftarrow 2$; $j \leftarrow n$

Repita

  enquanto $L[i] < v$ faça $i \leftarrow i + 1$

  enquanto $L[j] \geq v$ faça $j \leftarrow j - 1$

  se $i < j$ então Troca($L[i]$,$L[j]$)

Até $i > j$

$L[1] \leftarrow L[j]$; $L[j] \leftarrow v$

retorna(j,L)

Quicksort

Qual a complexidade de Particiona?

Quicksort

Qual a complexidade de Particiona?

$O(n)$

Exercício. Provar que complexidade de pior caso de Particiona é $O(n)$

Quicksort

```
Quicksort(L,1,p)
   k ← pivô(L)
   se k ≠ 0 então
      Troca(L[1],L[k])
      j,L ← Particiona(L,1,p)
      L1 ← Quicksort(L,1,j-1)
      L2 ← Quicksort(L,j,p)
      L  ← Concatena(L1,L2)
  retorna(L)
```

Qual a complexidade média de Quicksort, CM_Q?

Quicksort

Supor que *L* tem *n* elementos.

Temos:

- Escolha do pivô e particionamento
- Chamada do quicksort para a lista esquerda
- Chamada do quicksort para a lista direita

O desempenho depende de quem é o pivô.

Quicksort

Supor que $p(i)$ é a probabilidade do pivô ser o i+1-ésimo elemento

Então:

$$CM_Q(n) = \sum_{i=1}^{n-1} p(i)\left[CM_{Part}(n) + CM_Q(i) + CM_Q(n-i)\right]$$

O pivô será do 2º ao n-ésimo elemento

Complexidade média do particionamento

Quicksort

CM_{Part} é $O(n)$ ⟵ (Exercício. Provar)

Temos: $$CM_Q(n) = \sum_{i=1}^{n-1} p(i)\left[n + CM_Q(i) + CM_Q(n-i)\right]$$

Chamando CM_Q de T(n), temos:

$$T(n) = O\left(\sum_{i=1}^{n-1} p(i)\left[n + T(i) + T(n-i)\right]\right)$$

Quicksort

Supor todos os elementos diferentes, então o pivô será um dos dois primeiros elementos da lista, o maior deles.

Para calcular a probabilidade de que o (i+1)-ésimo elemento na ordenação esteja na primeira ou segunda posição da lista, devemos ter em mente que i elementos são menores que ele.

Quicksort

A probabilidade do (*i*+1)-ésimo elemento (ordenado) (1º, 2º, ...,nº) estar na primeira posição da lista é 1/*n* e a probabilidade do 2º ser menor

que ele é $\frac{i}{n-1}$

Quicksort

Assim a probabilidade de o (*i*+1)-ésimo elemento ser o maior e estar na primeira posição é:

$$\frac{1}{n}\frac{i}{n-1}$$

Que é também a probabilidade do (*i*+1)-ésimo elemento estar na segunda posição e ser o maior.

Quicksort

Assim a probabilidade de o (*i*+1)-ésimo elemento ser o pivô é:

$$\frac{2i}{n(n-1)}$$

Para $n > 1$, temos:

$$T(n) \text{ é } O\left(\sum_{i=1}^{n-1} \frac{2i}{n(n-1)}\left[n + T(i) + T(n-i)\right]\right)$$

Quicksort

Ou $T(n)\ é\ O\left(\frac{2}{n(n-1)}\sum_{i=1}^{n-1} i\left[n+T(i)+T(n-i)\right]\right)$

Sabe-se que

$$\sum_{i=1}^{n-1} f(i) = \sum_{i=1}^{n-1} f(n-i)$$

$$\sum_{i=1}^{n-1} f(i) = \frac{1}{2}\sum_{i=1}^{n-1} f(i) + f(n-i)$$

Quicksort

Tomando

$$f(i) = \frac{2i}{n(n-1)}\left[n + T(i) + T(n-i)\right]$$

Temos

$$f(n-i) = \frac{2(n-i)}{n(n-1)}\left[n + T(n-i) + T(i)\right]$$

Quicksort

$$\sum_{i=1}^{n-1} f(i) = \frac{1}{2}\left[\begin{array}{l}\displaystyle\sum_{i=1}^{n-1}\frac{2i}{n(n-1)}\left[n+T(i)+T(n-i)\right]+ \\ \displaystyle\sum_{i=1}^{n-1}\frac{2(n-i)}{n(n-1)}\left[n+T(n-i)+T(i)\right]\end{array}\right]$$

$$\sum_{i=1}^{n-1} f(i) = \frac{1}{(n-1)}\sum_{i=1}^{n-1}\left[n+T(i)+T(n-i)\right]$$

Quicksort

$$\sum_{i=1}^{n-1} f(i) = \frac{1}{(n-1)} \sum_{i=1}^{n-1} [n + 2T(i)]$$

$$\sum_{i=1}^{n-1} f(i) = n + \frac{2}{n-1} \sum_{i=1}^{n-1} [T(i)]$$

$$T(n) \acute{e}\ O(n) + O\left(\frac{2}{n-1} \sum_{i=1}^{n-1} [T(i)] \right)$$

Quicksort

Pode-se mostrar por indução em $n > 1$ que

$$\frac{2}{n-1}\sum_{i=1}^{n-1}[T(i)] \quad \acute{e} \quad O(n\log n)$$

Exercício. Mostrar

Portanto CM_Q é $O(n\log n)$.

Exercício

1. Determinar a expressão da complexidade média de uma busca não ordenada de *n* chaves, *n* par, em que as probabilidades de busca das chaves de ordem ímpar são iguais entre si, sendo esse valor igual ao dobro da probabilidade de qualquer chave par. Supor, ainda, que a probabilidade de a chave se encontrar na lista é igual a *q*.


FIM